串接指南：從其他系統讀取傳真（v0.6 起）

適用版本：v0.6.0-alpha

2026-10-27

目錄

- 1. 傳真的狀態
- 2. 取得 API 金鑰
- 3. 呼叫 API
- 4. 增量查詢（建議的串接方式）
- 5. 資料庫 view `fax_records`
- 6. 另一種方式：Webhook
- 7. 範例程式

給要把傳真接進自己系統（ERP、客服系統、客製程式）的開發人員。sip-faxd 提供兩種方式，可以並用：

方式	適合	拿得到
API（HTTPS，API 金鑰）	程式在別台機器；需要檔案或縮圖	狀態與資訊、TIFF / PDF、每頁 PNG（原圖、縮圖）；可選擇回寫已讀與備註
資料庫 view `fax_records`（唯讀帳號）	報表、BI、已經習慣直接查 DB 的系統	狀態與資訊（不含檔案）

檔案與縮圖**只能走 API**：資料庫裡的檔案路徑只在 sip-faxd 那台機器上有效，view 也不提供路徑。

範例程式：`examples/java`（JDK 17 以上，不需要任何套件）、`examples/node`（Node.js 18 以上，不需要任何套件）。完整的 API 規格：`docs/openapi.yaml`（OpenAPI 3.1），執行中的服務也在 `https://<主機>:8443/api/v1/openapi.yaml` 提供，可以直接拿去產生用戶端程式。

1. 傳真的狀態

每通來電一筆紀錄，`id` 不會重複、不會重用。

`fax_status`	意義	之後還會變嗎
`receiving`	通話進行中，`pages` 是目前已收到的頁數	會：通話結束時變成下面其中之一
`received`	完整收到	不會（已讀、備註、檔案清整除外）
`partial`	收到部分頁面（中途斷線等）	不會
`failed`	一頁都沒收到	不會
`no_fax`	接通了但沒有傳真訊號（例如有人打錯電話）	不會

- 檔案：`has_pdf` / `has_tiff` 為 true 時才能下載。保留期限到了檔案會被清掉（`files_purged_at` 有值，下載回 410 `purged`），紀錄本身依另一個保留期限保留。請在收到後就把需要的檔案存走。
- 會讓 `updated_at` 改變的事：接收中每多收一頁、通話結束、已讀 / 未讀、備註、檔案清整。增量查詢（第 4 節）就是依 `updated_at` 找出這些變動。
- 時間一律是 **UTC**（API 是 RFC 3339，例如 `2026-10-27T08:30:00.123Z`；資料庫見第 6 節），顯示時再轉成當地時間。

2. 取得 API 金鑰

由 sip-faxd 的管理員建立，每個串接的系統一把：

- Web 介面：管理員登入 →「API 金鑰」→「建立金鑰」。
- 或在伺服器上：`./sip-faxd apikey create erp`（可寫入已讀與備註時加 `--write`；有期限時加 `--expires 2027-12-31`）。

金鑰形如 `sfx_` 加 43 個字元，**只在建立時顯示一次**，伺服器只存它的雜湊；遺失時撤銷（Web 或 `./sip-faxd apikey revoke erp`）後重建。金鑰請放在設定檔或密碼庫，不要寫進程式碼或版本控制。

權限：

權限	能做的事
唯讀（`fax:read`，每把都有）	列表、增量查詢、單筆、計數、下載 TIFF / PDF、每頁 PNG
回寫（`fax:write`，建立時勾選）	另外可以標記已讀 / 未讀、修改備註；紀錄中的操作者是 `api:<金鑰名稱>`

任何金鑰都**不能**修改設定、帳號、投遞，也不能看投遞紀錄。管理員可以在「API 金鑰」頁看到每把金鑰的最後使用時間與來源 IP。

3. 呼叫 API

```
GET /api/v1/faxes/changes HTTP/1.1
Host: fax.example.com:8443
Authorization: Bearer sfx_xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
```

```
KEY=sfx_...                          # 從環境變數或密碼庫取得
BASE=https://fax.example.com:8443/api/v1
curl --cacert sip-faxd.crt -H "Authorization: Bearer $KEY" "$BASE/faxes?limit=20"
curl --cacert sip-faxd.crt -H "Authorization: Bearer $KEY" -o 42.pdf "$BASE/faxes/42/file.pdf"
curl --cacert sip-faxd.crt -H "Authorization: Bearer $KEY" -o 42-1.png "$BASE/faxes/42/pages/1.pn
        g?kind=thumb"
```

方法與路徑	說明
`GET /faxes/changes?after=&since=&limit=`	新進與有變動的傳真，最舊的變動在前（第 4 節）
`GET /faxes?status=&from=&to=&ani=&dnis=&q=&cursor=&limit=`	列表（最新的在前），可以篩選；`next_cursor` 翻頁
`GET /faxes/counts`	未讀數
`GET /faxes/{id}`	單筆
`GET /faxes/{id}/file.pdf` 、`/file.tif`	檔案
`GET /faxes/{id}/pages/{n}.png?kind=thumb`	第 n 頁（從 1 開始）的縮圖；`kind=full` 是原尺寸。支援 `ETag` / `If-None-Match`
`PUT /faxes/{id}/read` `{"read": true}`	已讀（需要回寫權限）
`PUT /faxes/{id}/note` `{"text": "...", "expected_updated_at": null}`	備註（需要回寫權限；`expected_updated_at` 填上次看到的 `note_updated_at`，別人改過時回 409）

錯誤一律是 `{"error": {"code": "...", "message": "..."}}`，程式請依 `code` 判斷：

HTTP	code	處理
400	`bad_request`	參數錯誤（未知參數也會被拒，避免打錯字卻回傳全部）
401	`invalid_api_key`	金鑰錯誤、已撤銷或已過期（不會區分是哪一種）。停止重試，通知管理員
403	`forbidden`	金鑰沒有回寫權限
404	`not_found`	沒有這筆、這頁或這個檔案
409	`conflict`	備註被別人改過：重新讀取後再改
410	`purged`	檔案已依保留期限清除
503	`unavailable`	資料庫暫時連不上：稍後重試

HTTPS 憑證

sip-faxd 預設用 HTTPS（port 8443）。

- **建議**：在 sip-faxd 的 `[web]` 設定公司自己 CA 簽的憑證（`cert` 、 `key` ），用戶端就不需要任何額外設定。
- 沒有設定時，sip-faxd 使用**自簽憑證**，約兩年自動更新一次，**更新後指紋會改變**，用戶端要重新匯入。取得方式：`openssl s_client -connect fax.example.com:8443 -showcerts </dev/null | openssl x509 > sip-faxd.crt` ，並與 sip-faxd 啟動 log 中的 SHA-256 指紋比對。
  - Java：`keytool -importcert -alias sip-faxd -file sip-faxd.crt -keystore fax-truststore.p12 -storetype PKCS12 -storepass changeit -noprompt` ，執行時加 `-Djavax.net.ssl.trustStore=fax-truststore.p12 -Djavax.net.ssl.trustStorePassword=changeit` 。
  - Node.js：環境變數 `NODE_EXTRA_CA_CERTS=sip-faxd.crt` 。
  - 不要關掉憑證檢查（例如 `NODE_TLS_REJECT_UNAUTHORIZED=0` ）：金鑰會被中間人取得。

4. 增量查詢（建議的串接方式）

`GET /faxes/changes` 依 `(updated_at, id)` 由舊到新回傳新進與有變動的傳真，並一定附上 `next` 。程式每隔一段時間（例如 10 秒）帶上一次的 `next` 當 `after` 呼叫，就會**每個變動恰好看到一次**；同一份傳真之後又變動（接收完成、被標已讀、備註修改）會再出現一次，請以 `id` 更新自己的資料。

```
cursor = 讀取上次存下的 cursor（第一次沒有）
loop:
    r = GET /faxes/changes?after=<cursor>&limit=100      （第一次可改用 since=<某個時間>）
    for fax in r.items:
        以 fax.id 新增或更新自己的紀錄
        if fax.fax_status != "receiving" 且 fax.has_pdf 且 還沒下載過:
            GET /faxes/{id}/file.pdf、/pages/1.png?kind=thumb
    cursor = r.next；存起來（處理完才存，當機重來只會重複、不會漏）
    if r.items 數量 < limit: 等 10 秒
```

- 最近 2 秒內的變動會延後到下一次才回傳，避免與正在寫入的資料競態而漏掉。
- `next` 是不透明的字串，請原樣保存，不要自己解析。
- 只要收完的傳真：略過 `fax_status = receiving` 的項目即可，通話結束時同一份傳真會再出現。

5. 資料庫 view `fax_records`

v0.6 起（migration 0004，啟動時自動套用）四種資料庫都有 view `fax_records`。**對外只承諾這個 view**：之後的版本只會增加欄位，不會改名或刪除；內部資料表 `faxes` 的結構可能改變，請不要直接查它。

欄位	型別（概念）	說明
`id`	整數	傳真編號，與 API 的 `id` 相同
`call_id`	字串	SIP Call-ID
`trunk`	字串	中繼名稱
`ani`	字串，可 NULL	來電號碼
`ani_source`	字串，可 NULL	號碼來源：`pai` 、`rpid` 、`from` 、`anonymous`
`dnis`	字串，可 NULL	撥打的號碼
`dnis_source`	字串，可 NULL	`ruri` 、`to` 、`diversion`
`remote_ident`	字串，可 NULL	對方傳真機的識別（TSI）
`started_at`	時間	來電時間
`answered_at`	時間，可 NULL	接通時間
`ended_at`	時間，可 NULL	掛線時間（接收中為 NULL）
`duration_sec`	整數，可 NULL	通話秒數
`hangup_direction`	字串，可 NULL	`remote` 、`local` 、`system`
`hangup_code`	字串，可 NULL	掛線原因代碼（`completed` 、`peer_hangup` 、`t30_failed` …）
`hangup_text`	字串，可 NULL	掛線原因說明
`sip_status`	整數，可 NULL	拒接時的 SIP 狀態碼
`t30_code`	整數，可 NULL	T.30 結果碼
`fax_status`	字串	第 1 節的狀態
`pages`	整數	頁數
`resolution`	字串，可 NULL	解析度
`bit_rate`	整數，可 NULL	傳輸速率（bit/s）
`compression`	字串，可 NULL	`mh` 、`mr` 、`mmr`
`ecm`	布林	是否使用 ECM
`modem`	字串，可 NULL	`v27ter` 、`v29` 、`v17`

欄位	型別（概念）	說明
`media`	字串，可 NULL	`t38` 、 `g711`
`has_tiff`	布林	TIFF 可以下載
`has_pdf`	布林	PDF 可以下載
`tiff_size`	整數，可 NULL	TIFF 大小（bytes）
`pdf_size`	整數，可 NULL	PDF 大小（bytes）
`files_purged_at`	時間，可 NULL	檔案被清除的時間
`is_read`	布林	已讀
`read_changed_by`	字串，可 NULL	誰改了已讀（帳號或 `api:<金鑰名稱>` ）
`read_changed_at`	時間，可 NULL	
`note`	字串，可 NULL	備註
`note_updated_by`	字串，可 NULL	
`note_updated_at`	時間，可 NULL	
`created_at`	時間	紀錄建立時間
`updated_at`	時間	最後變動時間（增量查詢用，有索引）

布林在 PostgreSQL 是 `boolean` 、MySQL / MariaDB 是 0 / 1、SQL Server 是 `bit` 、SQLite 是 0 / 1。

時間

所有時間都以 **UTC** 儲存，毫秒精度：PostgreSQL `timestamptz` ；SQL Server `datetime2(3)` ；MySQL / MariaDB `datetime(3)` （沒有時區資訊，內容是 UTC，請不要用連線時區轉換）；SQLite 是 RFC 3339 文字。

增量查詢

與 API 相同的做法，用 `(updated_at, id)` 當游標，並留 5 秒的安定期（資料庫主機與 sip-faxd 主機的時鐘可能差一點）：

```sql
-- PostgreSQL
SELECT * FROM fax_records
WHERE (updated_at > :last_updated OR (updated_at = :last_updated AND id > :last_id))
  AND updated_at <= now() - interval '5 seconds'
ORDER BY updated_at, id
LIMIT 100;

-- MySQL / MariaDB
SELECT * FROM fax_records
WHERE (updated_at > ? OR (updated_at = ? AND id > ?))
  AND updated_at <= UTC_TIMESTAMP(3) - INTERVAL 5 SECOND
ORDER BY updated_at, id
LIMIT 100;

-- SQL Server
SELECT TOP (100) * FROM fax_records
WHERE (updated_at > @last_updated OR (updated_at = @last_updated AND id > @last_id))
  AND updated_at <= DATEADD(second, -5, SYSUTCDATETIME())
ORDER BY updated_at, id;
```

第一次執行時 `last_updated` 用一個很早的時間（例如 `1970-01-01`）、`last_id` 用 0。每批處理完把最後一列的 `updated_at`、`id` 存起來。

建立唯讀帳號

請 DBA 以 sip-faxd 資料庫的擁有者身分執行（把 `sipfax` 換成實際的資料庫名稱、密碼換成強密碼）。這個帳號只能讀 `fax_records`，讀不到其他資料表（帳號、設定、金鑰），也不能寫入。

PostgreSQL：

```sql
CREATE ROLE fax_reader LOGIN PASSWORD 'change-me';
GRANT CONNECT ON DATABASE sipfax TO fax_reader;
GRANT USAGE ON SCHEMA public TO fax_reader;
GRANT SELECT ON fax_records TO fax_reader;
```

MySQL / MariaDB：

```sql
CREATE USER 'fax_reader'@'%' IDENTIFIED BY 'change-me';
GRANT SELECT ON sipfax.fax_records TO 'fax_reader'@'%';
```

SQL Server：

```
CREATE LOGIN fax_reader WITH PASSWORD = 'Change-me-1';
USE sipfax;
CREATE USER fax_reader FOR LOGIN fax_reader;
GRANT SELECT ON dbo.fax_records TO fax_reader;
```

SQLite 是 sip-faxd 本機上的檔案，不適合從別台機器讀取：請改用 API，或改用伺服器型資料庫（`sip-faxd db copy`，見 `docs/database.md`）。

6. 另一種方式：Webhook

如果希望「收到時主動通知」而不是定時查詢，可以在 sip-faxd 的投遞設定加一個 Webhook：收到傳真時 sip-faxd 會 POST 一份 JSON（可附 PDF），以 HMAC-SHA256 簽章（`X-Sipfax-Signature`），失敗會重試。設定與驗簽方式見 Web 介面「設定 → 投遞」的說明。Webhook 與 API 可以並用，例如以 Webhook 即時通知、以 `changes` 補漏。

7. 範例程式

檔案	內容
`examples/java/FaxClient.java`	JDK 17 以上，只用內建的 `HttpClient`：增量查詢、下載 PDF 與第 1 頁縮圖、保存 cursor。`java FaxClient.java` 直接執行
`examples/java/FaxDbReader.java`	JDBC 讀 `fax_records` 的增量查詢（需要對應資料庫的 JDBC driver）
`examples/node/fax-client.mjs`	Node.js 18 以上，只用內建的 `fetch`：與 Java 版相同

執行方式見 `examples/README.md`。